

Reflective Report: Ezypick

Peter Wanchai · Advanced iOS Development · Assessment 2

What the domain taught me

I started Assessment 2 assuming I would build what I pitched in Assessment 1: a swipe deck of cuisine and vibe cards ending on one restaurant. What changed my mind was a search that came back empty.

The first time, I thought I had written a bug. I turned halal and vegetarian on, searched, and got nothing back. The screen didn't tell me why, and it could have been any of three things: the filter, the hour (I work nights, so fewer places are open), or where I happened to be standing.

It was the filter. The data isn't there. Gluten-free has no field anywhere in the API. Halal is a category almost nobody publishes: one venue across the inner city. Vegetarian is the subtle one: the flag exists and almost never says no. Two venues in fifty-seven say false, and absent doesn't mean no, so vetoing on it ruled out two places while looking exactly like a safety check.

So I deleted the rule. It was the hardest one I had written, with its own type so a requirement could never be outvoted by a preference, and it was built on data that does not exist. The app now filters on the three things a phone can actually know: budget, walking distance, and whether the kitchen is open right now.

The smaller lesson was about the questions. Of the nine places a live search found around me, one was Thai, so "Feeling Italian?" rules out almost nothing while "Got time to sit down properly?" splits the same set roughly in half. I picked the swipe deck because it looked good as an interaction, not because it decided anything. The app now writes its questions from the restaurants still in the running.

Why these use cases

I put the rules in use cases so the view is decoupled from them. Presentation and logic should be separated when they can be, and it means every rule can be tested without launching the app.

`AskNextQuestionsUseCase` is the one that earned it. A language model writes the questions, but the use case checks each one against the restaurants actually left and discards it if it would not narrow anything or has already been asked. That rule caught something I could not see: for a while the model was answering on every run and not one of its questions was usable, so the app quietly fell back to its built-in ones and looked exactly like a success. The model phrases; it never decides.

`ShortlistRestaurantsUseCase` holds the hard limits and runs before a single question is asked, so nothing unaffordable or shut ever reaches the part of the app that is less careful. `AnswerQuestionUseCase` refuses an answer that would leave the diner with nowhere to eat, and `SaveDiningPreferencesUseCase` refuses a profile that could never return a result.

The fifth, `SurveyNearbyRestaurantsUseCase`, holds no rule at all, and that is the point. It reports how many places are nearby before anything is removed. Someone told "one place fits" would think the neighbourhood was empty rather than their budget tight.

Where a person can go wrong

Someone sets their walk to five minutes on a rainy day and forgets. Every lunch after that is quietly starved of options, and they blame the app rather than the setting.

That matters because deciding where to eat is underestimated. It takes mental energy, and Alex ends up at the same place and misses out.

So the filtering step counts what each limit excluded, and the failure names its own cause: "the places that fit are all further than you said you'd walk", instead of "no results".

My first version of that recovery was itself broken. The screen named the cause correctly and then offered one button, "Spend a bit more today". Budget causes one refusal in four; on the other three it lifted a cap that was not the problem and returned the identical screen. I only found it by tapping it. It says "Adjust settings" now and opens the profile editor, so whatever the screen just blamed is where you go to change it.

What I would do next

Group lunches. When there are more people it is harder to decide, so that is where an app helps most.

I cut it deliberately, because I would rather ship one honest single-user version than a half-working group one. The architecture was built to take it: a lunch becomes a party of several voters, the same five use cases run, and answers are tallied instead of applied. Only the stopping rule changes, and it changes by vanishing, because one person is already a party of one.